

第19回 JavaScript の基本 4

実習時のフォルダ構成

第 18 回と同じ形式で、Web_名前 の中に 19 フォルダを作る。19 の中に sample と exercise を作り、次のファイルを用意する。

```
Web_名前
├── 19
│   ├── sample
│   │   ├── index.html
│   │   └── main.js
│   └── exercise
│       ├── index.html
│       └── main.js
```

- sample：講義で使うサンプル（講義内容のコードは、すべて sample/main.js に書く）
- exercise：演習で作るファイル（演習 1～4、発展）

sample/index.html と exercise/index.html には、どちらも次の内容を書く（第 18 回の sample/index.html をコピーして、<title> だけ変えてもよい）。main.js を読み込んで、ブラウザで実行できる最小限の HTML である。

```
<!DOCTYPE html>
<html lang="ja">
<head>
  <meta charset="UTF-8">
  <title>JavaScriptの基本 4</title>
</head>
<body>
  <script src="main.js"></script>
</body>
</html>
```

講義内容

以降の講義内容のコードは、sample/main.js に、上から順に書き足していく。書いたら sample/index.html をブラウザで開き、F12 キーの開発者ツール（コンソール）で結果を確認する。

1. 前回の復習

- if (条件) { }：条件が true のときだけ、{ } の中を実行する。
- else・else if で、条件が false のときの処理や、3 つ以上の分かれ道を書ける。

```
const hp = 0;
if (hp <= 0) {
  console.log("ゲームオーバー");
}
```

- 今回は、同じ処理を何回も実行する「繰り返し」と、たくさんの値をまとめて入れる「配列」を学ぶ。

2. for 文

同じ処理を決まった回数だけ繰り返すには、`for` 文を使う。

```
for (初めの準備; 続ける条件; 1 回ごとの変化) {
  繰り返す処理
}
```

```
for (let i = 0; i < 5; i++) {
  console.log(i);           // 0 1 2 3 4
}
```

コンソールには、`0` から `4` までが 1 行ずつ表示される。次の順で動いている。

回	<code>i</code> の値	条件 <code>i < 5</code>	実行すること
1 回目	0	<code>true</code>	<code>0</code> を表示 → <code>i++</code> で <code>i</code> が 1 になる
2 回目	1	<code>true</code>	<code>1</code> を表示 → <code>i</code> が 2 になる
...	...	...	...
5 回目	4	<code>true</code>	<code>4</code> を表示 → <code>i</code> が 5 になる
(終了)	5	<code>false</code>	繰り返しを終わる

- `let i = 0` : 最初に 1 回だけ実行する。数を数えるための変数 `i` を用意する。
- `i < 5` : 毎回、繰り返す前に調べる。`true` の間は繰り返し、`false` になったら終わる。
- `i++` : 1 回繰り返すごとに実行する。`i++` は `i += 1` と同じ (1 増やす)。
- `i` は、繰り返しの回数を数える変数によく使う名前。`for` の `()` の中で作った `i` は、その `for` の中でだけで使える。そのため、この授業では、次の `for` でも同じ名前 `i` を使う。
- `{ }` の中は、`if` 文と同じく、スペース 2 つ分の字下げをして書く。

`for` 文の中で、変数の値を少しずつ変えることもできる。キャラクターを右へ動かす例。

```
let playerX = 0;
for (let i = 0; i < 5; i++) {
  playerX += 10;           // 右へ 10 動く
  console.log(playerX);    // 10 20 30 40 50
}
```

- ゲームでは、このように「少しだけ動かす」処理を何回も繰り返して、キャラクターを動かす。

3. i-- で数を減らしながら繰り返す

i-- は i -= 1 と同じ (1 減らす)。大きい数から小さい数へ、逆向きに繰り返せる。

```
for (let i = 3; i >= 1; i--) {  
  console.log(i);           // 3 2 1  
}  
console.log("スタート!");
```

- i は 3 から始まり、i >= 1 の間、1 ずつ減りながら繰り返す。
- console.log("スタート!"); は { } の外にあるので、繰り返しが終わってから 1 回だけ実行される。

4. 配列

たくさんの値を、1 つの変数にまとめて入れたいときは、「配列」を使う。[] の中に、値を、で区切って並べる。

```
const enemyNames = ["スライム", "ゴブリン", "ドラゴン"];  
console.log(enemyNames[0]);           // スライム  
console.log(enemyNames[2]);           // ドラゴン  
console.log(enemyNames.length);       // 3  
console.log(enemyNames[enemyNames.length - 1]); // ドラゴン (最後の要素)  
console.log(enemyNames[3]);           // undefined (番号 3 の要素はない)
```

番号	0	1	2
値	"スライム"	"ゴブリン"	"ドラゴン"

- 配列の中の 1 つ 1 つの値を「要素」という。
- 要素は、配列[番号] で取り出す。番号は 0 から始まる。
- 配列.length で、要素の数が分かる。最後の要素の番号は、length - 1。
- ない番号を指定すると、undefined (値がない) になる。

要素は、番号を指定して書き換えられる。

```
let enemyHps = [30, 50, 80];  
enemyHps[1] = 20;           // 番号 1 の要素を書き換える  
console.log(enemyHps);       // [30, 20, 80]
```

- この回では、要素を書き換える配列には let を使う。const で作った配列についての詳しい扱いは、次のオブジェクトとあわせて学ぶ。
- ブラウザのコンソールでは、(3) [30, 20, 80] のように、要素の数も一緒に表示されることがある。

5. for 文で配列の要素を順に処理する

for 文の i を配列の番号として使うと、すべての要素を順に処理できる。

```
for (let i = 0; i < enemyNames.length; i++) {
  console.log(`${i}番: ${enemyNames[i]}`); // 0番: スライム 1番: ゴブリン 2番: ドラゴン
}
```

- 条件は `i < enemyNames.length` (要素の数より小さい間) にする。 `<=` にすると、ない番号 (3) まで調べてしまう。
- `length` を使うと、配列の要素の数が変わっても、そのまま使える。

すべての要素の値を変える例。3 発の弾を、まとめて上へ動かす。

```
let bulletYs = [400, 300, 200]; // 3 発の弾の y 座標
for (let i = 0; i < bulletYs.length; i++) {
  bulletYs[i] -= 50; // すべての弾を上へ 50 動かす
}
console.log(bulletYs); // [350, 250, 150]
```

- 画面の y 座標は、上へ行くほど小さくなる (Canvas の回で詳しく学ぶ)。そのため、上へ動かすときは `-=` を使う。

すべての要素を足し合わせる例。

```
let totalHp = 0;
for (let i = 0; i < enemyHps.length; i++) {
  totalHp += enemyHps[i];
}
console.log(totalHp); // 130
```

- 合計を入れる変数は、`for` の前に `0` で用意しておく。

`i--` を使うと、配列を後ろから順に処理できる。

```
for (let i = enemyNames.length - 1; i >= 0; i--) {
  console.log(enemyNames[i]); // ドラゴン ゴブリン スライム
}
```

- 最後の要素の番号 (`length - 1`) から始めて、番号 0 まで 1 ずつ減らす。

6. for 中の if

`for` 文の `{ }` の中に、`if` 文を書ける。「すべての要素のうち、条件に合うものだけ」を処理できる。

```
const shotYs = [120, 30, -20, 60, -5]; // 弾の y 座標 (マイナスは画面の上の外)
let outCount = 0;
for (let i = 0; i < shotYs.length; i++) {
  if (shotYs[i] < 0) {
    console.log(`${i}番の弾は画面の外`); // 2番の弾は画面の外 4番の弾は画面の外
    outCount += 1;
  }
}
console.log(`画面の外の弾: ${outCount}個`); // 画面の外の弾: 2個
```

- `if` の `{ }` は、`for` の `{ }` の中にあるので、さらにスペース 2 つ分、字下げする。

- 条件に合った数を数えるときは、数える変数を `for` の前に `0` で用意し、`if` の中で `+= 1` する。

7. 最終回のゲームとのつながり

今回学んだことは、最終回（第 30 回）に完成させるシューティングゲームで、次のように使う。

今回学んだこと	最終回のシューティングゲームでの使い道
<code>for (let i = 0; i < 配列.length; i++)</code> で、すべての要素を処理する	画面にあるすべての弾・敵を描く
すべての弾の <code>y</code> 座標を <code>-= 50</code> で動かす	すべての弾を上へ、すべての敵を下へ動かす
<code>i--</code> と <code>length - 1</code> で、配列を後ろから処理する	弾や敵を調べて、いらなくなったものを消す（第 24 回で学ぶ）
<code>for</code> の中の <code>if</code> (<code>y</code> 座標がマイナスなら画面の外)	画面の外に出た弾・敵を見つける
<code>if</code> の中で <code>+= 1</code> する（条件に合ったら数える）	敵に弾が当たったら得点を増やす。敵にぶつかったら残機を減らす
<code>for</code> の中で <code>x</code> 座標を少しずつ増やす／画面の端で止める（演習 2）	自機を少しずつ動かし、画面の端で止める（第 22 回からは、 <code>for</code> の代わりに「ゲームループ」で繰り返す）
要素を書き換える配列には <code>let</code> を使う	弾・敵の配列。リスタートのときに、空の配列に戻す
条件は <code><</code> にする（ <code><=</code> だとない番号まで調べてしまう）	すべての繰り返し。間違えると、エラーの原因になる

- 今回は、弾を「`y` 座標の数値だけ」の配列で表した。最終回のゲームでは、1 つの弾が位置と大きさ（`x` 座標・`y` 座標・幅・高さ）をまとめて持つ。この書き方は、第 20 回（オブジェクト）と第 24 回（弾の発射）で学ぶ。
- 配列に要素を追加する方法、配列から要素を消す方法は第 24 回で、`for` の中に `for` を書く方法は第 26 回で学ぶ。

演習

`exercise/main.js` に書く。結果は、`exercise/index.html` をブラウザで開いて、`F12` の開発者ツール（コンソール）で確認する。

演習 1 `for` 文で数を表示する

- `for` 文と `i++` を使って、1 から 10 までを表示する。
- `for` 文と `i--` を使って、5 から 1 までを表示し、最後に「GO!」と表示する。

演習 2 `for` 文で自機を動かす

- 画面の幅（100）と自機の速さ（15）を `const` で、自機の `x` 座標（0）を `let` で用意する。

- `for` 文で 10 回繰り返し、毎回、x 座標に速さを足す。
- x 座標が画面の幅より大きくなったら、画面の幅の値に戻す（第 18 回の「画面の端で止める」）。
- 毎回、「○回目：x = △」と表示する（○は 1～10）。途中から x 座標が 100 で止まることを確認する。

演習 3 配列を作って表示する

- 好きなアイテムの名前を 4 つ入れた配列を、`const` で作る。
- 最初の要素と、最後の要素を表示する。最後の要素は、`length` を使って取り出す。
- 「アイテムの数：○」と表示する。
- `for` 文で、すべてのアイテムを「1：○○」「2：○○」…の形で表示する（番号は 1 から）。

演習 4 `for` の中の `if`

- 5 体の敵の HP を入れた配列を作る（例：[0, 25, 0, 40, 10]）。
- `for` 文で順に調べ、HP が 0 以下の敵は「○番の敵：倒した」、それ以外は「○番の敵：残りHP △」と表示する。
- 倒した敵の数を数えて、最後に「倒した敵：○体」と表示する。

発展（任意）

- 4 発の弾の y 座標を入れた配列（例：[300, 120, 40, 200]）を作る。`for` 文で、すべての弾を上へ 50 動かし、y 座標が 0 より小さくなった弾は「○番の弾は画面の外」と表示する。最後に配列を表示する。
- 演習 3 の配列を、`i--` を使って後ろから順に表示する。
- 演習 4 の敵の HP の合計を求めて表示する。